AGRI-INTELLIGENCE

Smart Irrigation Dashboard

User & Developer Guide

Frontend : React 19 + Vite + Tailwind CSS 4

Backend : Python FastAPI + Uvicorn

Database : Supabase (Auth + Storage)

Desktop : Electron 43

Hardware : ESP32 sensor nodes + ESP32-CAM

Goho Rakangwara Agri-Intelligence | October 2026

1. Project Overview

AGRI-INTELLIGENCE is a full-stack smart-irrigation platform built for smallholder and commercial farms. It combines real-time sensor data (soil moisture, flow, pressure, tank level, temperature/humidity) with AI-powered crop health analysis and WhatsApp alerts into a single operational hub.

The system consists of three main layers:

- Frontend Dashboard -- React 19 + Vite + Tailwind CSS 4 web app running on http://localhost:5173 in development, or packaged as a self-contained Electron desktop application.
- Backend API -- Python FastAPI served by Uvicorn on port 8021. Handles authentication (Supabase JWT), irrigation control, AI diagnosis, WhatsApp alerts via Twilio, and ESP32 hardware bridging.
- Hardware Layer -- ESP32 sensor/actuator nodes report soil-moisture readings and accept irrigation commands. An ESP32-CAM periodically photographs each bed for AI crop-health checks.

Technology Stack

Variable / Key	Value / Description
Frontend framework	React 19 with react-router-dom v7
Build tool	Vite 8 (+ @vitejs/plugin-react)
Styling	Tailwind CSS 4 via @tailwindcss/vite
Desktop wrapper	Electron 43
Charts	Recharts 3 + Leaflet / react-leaflet
Backend	FastAPI + Uvicorn
Auth	Supabase Auth (ES256 / RS256 JWT)
AI / Vision	Google Gemini (google-genai)
Notifications	Twilio WhatsApp + in-app alerts
Hardware	ESP32 nodes (HTTP API over LAN or tunnel)

2. Prerequisites

Before running the dashboard, ensure the following tools are installed:

- Node.js >= 18 (npm comes bundled)
- Python >= 3.11
- Git (optional, for cloning)
- A Supabase project (free tier works fine for development)
- A Google Gemini API key (optional -- required only for AI crop-health checks)
- A Twilio account (optional -- required only for WhatsApp alerts)

TIP

Use a Python virtual environment (.venv) to keep backend dependencies isolated from your global Python installation. The project ships with a backend/venv folder; activate it before running any pip commands.

3. Environment Setup

3.1 Backend -- .env file (backend/.env)

Copy backend/.env.example to backend/.env and fill in your credentials. Only SUPABASE_URL is strictly required to start the server.

Variable / Key	Value / Description
SUPABASE_URL	Your Supabase project URL
GEMINI_API_KEY	Google AI Studio API key (AI diagnosis)
ESP32_BASE_URL	ESP32 node LAN IP or tunnel URL (optional)
ESP32_SHARED_SECRET	Shared secret matching the .ino firmware
CAMERA_SHARED_SECRET	Shared secret for ESP32-CAM frames
SUPABASE_SERVICE_ROLE_KEY	Service role key for private storage bucket
TWILIO_ACCOUNT_SID	Twilio account SID (WhatsApp alerts)
TWILIO_AUTH_TOKEN	Twilio auth token
TWILIO_WHATSAPP_FROM	Sender number (e.g. +14155238886)
TWILIO_WHATSAPP_TO	Your phone in E.164 format
CORS_EXTRA_ORIGINS	Extra CORS origins, comma-separated (optional)

3.2 Frontend -- .env file (irrigation-dashboard/.env)

Copy irrigation-dashboard/.env.example to irrigation-dashboard/.env and fill in the values below.

Variable / Key	Value / Description
VITE_API_URL	http://localhost:8021 (default if unset)
VITE_SUPABASE_URL	Same Supabase project URL as the backend
VITE_SUPABASE_PUBLISHABLE_KEY	Supabase anon/publishable key (safe in browser)

SECURITY

Never put the Supabase service_role key in any VITE_* variable. Vite inlines all VITE_* variables into the built JavaScript bundle which is sent to every browser visitor.

4. Running the Dashboard via Terminal

4.1 Start the Backend API

Open a terminal window, navigate to the backend directory, activate the virtual environment, and launch the FastAPI server with Uvicorn:

```
# 1. Navigate to the backend folder
cd C:\Users\Bhamu\Desktop\AGRI-INTELLIGENCE\backend

# 2. Activate the virtual environment (Windows PowerShell)
.\venv\Scripts\Activate.ps1

# 3. Install dependencies (first time only)
pip install -r requirements.txt

# 4. Start the API server
uvicorn main:app --host 0.0.0.0 --port 8021 --reload
```

The server starts at http://localhost:8021. The --reload flag enables hot-reloading on code changes (development only). Visit http://localhost:8021 in a browser to confirm the API is running -- you should see: {"status": "Smart Irrigation API is running"}

4.2 Start the Frontend (Web / Browser Mode)

Open a second terminal window, navigate to the irrigation-dashboard folder, install Node packages (first time only), and start the Vite dev server:

```
# 1. Navigate to the dashboard folder
cd C:\Users\Bhamu\Desktop\AGRI-INTELLIGENCE\irrigation-dashboard

# 2. Install Node dependencies (first time only)
npm install

# 3. Start the Vite development server
npm run dev
```

Vite prints a local URL such as http://localhost:5173 (or 5174 if 5173 is occupied). Open that URL in any modern browser. The dashboard hot-reloads on every file save -- no manual refresh needed. Because vite.config.js sets host: true, the dev server is also reachable from any device on the same Wi-Fi network (useful for testing the mobile app).

4.3 Start the Desktop App (Electron)

With the Vite dev server already running on port 5173, open a third terminal and launch Electron:

```
cd C:\Users\Bhamu\Desktop\AGRI-INTELLIGENCE\irrigation-dashboard

# Run the Electron desktop wrapper
npm run electron
```

Electron opens a 1280×800 window loading http://localhost:5173. For an exhibition or kiosk deployment, set fullscreen: true and kiosk: true in electron/main.cjs before running this command.

4.4 Production Build

To build an optimised static bundle for deployment (Vercel, Netlify, Render, or a self-hosted server):

```
cd C:\Users\Bhamu\Desktop\AGRI-INTELLIGENCE\irrigation-dashboard

# Build the production bundle into dist/
npm run build

# Preview the production build locally
npm run preview
```

NOTE

For Electron production packaging change win.loadURL(...) in electron/main.cjs to win.loadFile('dist/index.html') and then run npm run build before npm run electron.

4.5 Quick-Start Cheat-Sheet

Variable / Key	Value / Description
Backend start	uvicorn main:app --host 0.0.0.0 --port 8021 --reload
Frontend dev	npm run dev
Electron app	npm run electron
Production build	npm run build
Preview build	npm run preview
Lint check	npm run lint

5. Dashboard Pages & Features

The dashboard is a single-page application with a collapsible sidebar. Each section is accessible from the sidebar or via its URL path.

Dashboard (/)

Operational home page. Displays live system-status strip, four headline KPI cards (water used, water saved, avg soil moisture, battery %), daily water-use chart, per-bed moisture panels, energy summary, farmer advisory and recent alerts. All data refreshes automatically every few seconds.

Goho AI (/ai)

Conversational AI assistant (powered by Google Gemini). Ask questions about farm conditions, get crop recommendations, and receive irrigation scheduling advice in natural language.

Crop Health (/crop-health)

AI-powered plant-disease and stress detection. The ESP32-CAM photographs each bed on a schedule or after every irrigation cycle. Results show condition, severity percentage, confidence score, and recommended treatment.

Irrigation Control (/irrigation)

Manual and automatic irrigation management. Start or stop irrigation for any bed, set cycle duration (1-120 minutes), view irrigation overview charts, and review season summary statistics.

Energy Management (/energy)

Live energy dashboard. Shows active power source (solar/battery/grid/hybrid), battery state of charge, solar panel output, hourly power consumption chart, CO2 savings, and grid changeover simulation.

Bed Monitoring (/beds)

Detailed per-bed view. Soil VWC trend chart, pump and valve status, temperature, humidity, soil temperature, last irrigation timestamp, water used per bed, and live bed-camera feed.

Aerial View (/aerial-view)

Interactive Leaflet map showing farm plot boundaries, bed locations, and sensor node positions. Click a marker for real-time sensor readings.

Data & Reports (/reports)

Exportable seasonal reports covering irrigation events, energy consumption, estimated crop yield, and environmental impact (CO2 avoided). Filter by date range and download as PDF or CSV.

Alerts & Notifications (/alerts)

Centralised alert feed. Filters by category (Bed Monitoring, Irrigation Control, Weather Forecast). Severity levels: critical, warning, info, success. Alerts also delivered via WhatsApp.

System Settings (/settings)

Editable configuration for the entire rig. Sections: Irrigation (mode, VWC thresholds, duration limits), Water (tank levels, flow thresholds, leak detection), Energy (power-source priority, battery thresholds), Notifications (event types and channels). Settings are persisted to backend/system_settings.json so they survive a server restart.

6. Backend API Reference

The FastAPI backend exposes a REST API on http://localhost:8021. All /api/* endpoints require a valid Supabase JWT in the Authorization: Bearer <token> header. Interactive Swagger docs are available at http://localhost:8021/docs.

`GET /`	Health check -- returns API running status (no auth required)
`GET /api/dashboard`	Full dashboard data snapshot (beds, KPIs, charts, alerts, ...)
`POST /api/irrigation/trigger/{bed_id}`	Instantly trigger a simulated irrigation cycle on a bed
`POST /api/irrigation/start/{bed_id}`	Start a timed irrigation cycle (body: {durationMin: 1-120})
`POST /api/irrigation/stop/{bed_id}`	Stop the active irrigation cycle on a bed
`GET /api/settings`	Read current system settings + device / sensor status
`PUT /api/settings`	Update system settings (full or partial JSON patch)
`POST /api/maintenance/{action}`	Run a maintenance action (restart-nodes, check-sensors, ...)
`GET /api/camera/latest/{bed_id}`	Fetch the latest camera image URL for a bed
`POST /api/camera/capture/{bed_id}`	Request an on-demand photo from the bed camera
`POST /api/diagnose/{bed_id}`	Run AI crop-health diagnosis on the latest camera frame

7. Hardware Integration

The backend bridges the web dashboard to physical ESP32 nodes. When ESP32_BASE_URL is set in backend/.env, Bed 1 uses real hardware; Beds 2 and 3 remain simulated.

7.1 ESP32 Sensor Node API (Bed 1)

Variable / Key	Value / Description
`GET /irrigate?sec=N`	Start irrigation for N seconds
`GET /stop`	Stop irrigation immediately
`GET /status`	JSON status: {state, remainingSec, vwc, sensorOk, auto}

All requests must include ?key=<ESP32_SHARED_SECRET> to prevent unauthorised access over a public tunnel. The backend polls /status at most once every 4 seconds to avoid overloading the single-threaded ESP32 WebServer.

7.2 ESP32-CAM (Bed Camera)

The camera firmware POSTs JPEG frames to the backend /camera/frame endpoint with the X-Camera-Key header. Frames are stored in a private Supabase Storage bucket and retrieved via signed URLs. AI diagnosis runs automatically after every irrigation cycle.

7.3 Automatic vs Manual Irrigation Mode

- AUTOMATIC -- The ESP32 node monitors soil VWC and starts irrigation when it drops below the configured trigger threshold. The backend detects the state change and logs it to Recent Activity.
- MANUAL -- Irrigation is started from the dashboard or mobile app. The backend translates the request into an ESP32 /irrigate?sec=N call.

8. Notifications & Alerts

The system delivers alerts through two channels: the in-dashboard Alerts page and WhatsApp via Twilio.

8.1 WhatsApp Alerts (Twilio)

Alerts are sent automatically for the following events:

- Soil moisture drops below the VWC trigger threshold
- Automatic irrigation cycle starts
- AI crop-health diagnosis finds a moderate or high-severity issue
- Manual irrigation triggered from the dashboard

8.2 Configuring Alert Channels

Open System Settings -> Notifications in the dashboard to toggle each event type and channel (WhatsApp, SMS, email, in-app) independently. Changes are persisted to the backend immediately.

NOTE

SMS and email channels are defined in the settings model but their transport implementation is pending. Only WhatsApp (Twilio) and in-app notifications are active in the current release.

9. System Settings Reference

All settings are accessible at GET /api/settings and editable via PUT /api/settings. They are persisted to backend/system_settings.json.

9.1 Irrigation Settings

Variable / Key	Value / Description
mode	automatic | manual
vwcTrigger	Irrigate below this VWC % (default 25)
vwcOptimalMin	Optimal band lower bound % (default 40)
vwcOptimalMax	Optimal band upper bound % (default 60)
maxDurationMin	Maximum single-cycle duration in minutes (default 45)
minIntervalMin	Minimum gap between cycles in minutes (default 30)
rainfallSkipEnabled	Skip cycle if rain forecast/measured (true/false)
rainfallSkipMm	Rain threshold in mm to skip a cycle (default 5)

9.2 Water / Pump Settings

Variable / Key	Value / Description
tankMinPct	Warn when tank drops below this % (default 20)
tankCriticalPct	Block irrigation below this % (default 10)
flowThresholdLpm	Nominal expected flow rate L/min (default 12)
minExpectedFlowLpm	Below this -> suspected blockage (default 6)
maxExpectedFlowLpm	Above this -> suspected burst / leak (default 20)
pressureMinBar	Minimum acceptable pressure bar (default 1.0)
pressureMaxBar	Maximum acceptable pressure bar (default 3.5)
leakDetectionEnabled	Continuously monitor for leaks (true/false)
dryRunProtectionEnabled	Stop pump if no flow detected (true/false)
pumpMaxRuntimeMin	Hard stop for continuous pump running (default 120)

9.3 Energy Settings

Variable / Key	Value / Description
powerSource	solar | battery | grid | hybrid
solarPriority	Prefer solar over grid (true/false)
gridBackup	Allow grid as backup power (true/false)
automaticTransfer	Auto-switch sources on failure (true/false)
lowBatteryPct	Issue low-battery alert below this % (default 20)
criticalBatteryPct	Issue critical alert below this % (default 10)
chargingPriority	Charge via solar or grid first

10. Per-Crop Soil-Moisture Targets

Each bed's VWC thresholds are tuned per crop type. These override the system-wide defaults and are defined in backend/main.py (CROP_TARGETS dict).

Variable / Key	Value / Description
`Covo (leafy brassica)`	Trigger 32% | Optimal 48-68%
`Tsunga (leafy brassica)`	Trigger 30% | Optimal 45-65%
`Tomato (deep-rooted)`	Trigger 22% | Optimal 34-52%

Covo and Tsunga are shallow-rooted and need consistently moist soil, hence higher trigger and optimal values. Tomato is deeper-rooted and more drought-tolerant; a mild dry-down before irrigating improves fruit set and reduces cracking.

11. Troubleshooting

Dashboard shows no data / 'failed to fetch'

Confirm the backend is running on port 8021. Check that VITE_API_URL in irrigation-dashboard/.env points to the correct address. If the backend is on a different machine, ensure CORS_EXTRA_ORIGINS includes the dashboard's origin.

Login page loops or 'invalid JWT'

Verify SUPABASE_URL in backend/.env matches the project URL in VITE_SUPABASE_URL. Ensure the user's Supabase account is active and the anon key is the publishable (not service-role) key.

Electron window is blank

The Vite dev server must be running on port 5173 before launching Electron. If Vite picked a different port, update the loadURL call in electron/main.cjs.

ESP32 node shows as offline

Check that ESP32_BASE_URL is the correct LAN IP or tunnel URL (no trailing slash). Confirm the SHARED_SECRET in the firmware sketch matches ESP32_SHARED_SECRET in backend/.env.

WhatsApp alerts not being sent

All four Twilio env vars must be set: TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_WHATSAPP_FROM, TWILIO_WHATSAPP_TO. The destination number must have joined the Twilio sandbox first.

npm install fails on Windows

Run PowerShell as Administrator and execute: Set-ExecutionPolicy RemoteSigned -Scope CurrentUser. Then retry npm install.

pip install fails (permissions)

Activate the virtual environment first: .\venv\Scripts\Activate.ps1. Never pip-install into the system Python on Windows.

12. Project Structure

```
AGRI-INTELLIGENCE/
|-- backend/                FastAPI backend
|   |-- main.py             API routes, bed state, ESP32 bridge
|   |-- auth.py             Supabase JWT verification
|   |-- camera.py           ESP32-CAM frame ingestion + AI diagnosis hook
|   |-- diagnosis.py        Google Gemini crop-health diagnosis
|   |-- requirements.txt    Python dependencies
|   |-- .env.example        Environment variable template
|   \-- system_settings.json  Persisted settings (auto-created)
|-- irrigation-dashboard/   React + Vite web app
|   |-- src/
|   |   |-- pages/          Route-level page components
|   |   |-- components/     Reusable UI widgets
|   |   |-- hooks/          Custom React hooks (data fetching)
|   |   |-- lib/            API client, Supabase, settings API
|   |   \-- routes.js       Route definitions (single source of truth)
|   |-- electron/main.cjs   Electron desktop wrapper
|   |-- vite.config.js      Vite build config
|   \-- .env.example        Frontend env template
|-- mobile-app/             React Native (Expo) mobile app
|-- firmware/               ESP32 Arduino sketches
|   |-- sensor-node/        Soil + pump control firmware
|   \-- camera-node/        ESP32-CAM streaming firmware
|-- architecture-diagrams/ System architecture PNGs
\-- docs/                   Documentation (this file)
```

13. Quick Reference Card

Tear-out card -- keep this handy when operating the system.

Terminal Commands

```
# --- BACKEND ---
cd AGRI-INTELLIGENCE\backend
.\venv\Scripts\Activate.ps1
uvicorn main:app --host 0.0.0.0 --port 8021 --reload

# --- FRONTEND ---
cd AGRI-INTELLIGENCE\irrigation-dashboard
npm install        # first time only
npm run dev        # browser at http://localhost:5173

# --- ELECTRON DESKTOP ---
npm run electron   # (while Vite dev server is running)

# --- PRODUCTION ---
npm run build      # outputs to dist/
npm run preview    # preview at http://localhost:4173
```

Default Ports

Variable / Key	Value / Description
Backend API	http://localhost:8021
Vite dev server	http://localhost:5173 (or 5174+)
Vite preview	http://localhost:4173
API docs (Swagger)	http://localhost:8021/docs
API docs (Redoc)	http://localhost:8021/redoc

Dashboard Routes

Variable / Key	Value / Description
/	Dashboard -- operational overview
/ai	Goho AI assistant
/crop-health	AI crop-health & disease detection
/irrigation	Irrigation control & history
/energy	Energy management & solar
/beds	Per-bed monitoring & camera
/aerial-view	Interactive farm map (Leaflet)
/reports	Seasonal data & exportable reports

`/alerts`	Alerts & notifications feed
`/settings`	System settings & device status